

Project Design Phase
Solution Architecture

Date	13 July 2026
Team ID	SWTID-2026-7108
Project Name	BookStore
Maximum Marks	4 Marks

Solution Architecture: 1. High-Level Architecture Component Breakdown

Your MERN stack architecture bridges the user experience across all devices with a robust, scalable backend:

- **Presentation Layer (Frontend - React.js):** A fully responsive UI that serves browsers on computers, tablets, and smartphones. It manages client-side routing, user registration, the interactive catalog, review submission, and purchase/reservation workflows.
- **Application Layer (Backend - Node.js & Express.js):** Rest APIs that handle incoming business logic, process data requests, manage user authentication (via JWT), handle recommendations, and communicate securely with the database.
- **Data Layer (Database - MongoDB):** A document-based database storing book inventory data (genres, authors, languages), user credentials, active book reservations, transaction logs, and user reviews.

2. Tech Stack & Integration Specifications

- **Client Devices:** Desktop, Tablet, and Mobile Web Browsers.
- **Frontend Framework:** React.js (with Axios for HTTP requests and Tailwind CSS or Bootstrap for responsive styling).
- **Server Environment:** Node.js runtime.
- **Web Framework:** Express.js.
- **Database:** MongoDB (hosted on MongoDB Atlas or locally, managed via Mongoose ODM).
- **Authentication:** JSON Web Tokens (JWT) for secure, stateindependent sessions.

3. Architecture and Data Flow

The text below traces the end-to-end data flow when a customer interacts with the platform (e.g., browsing, reading reviews, buying, or reserving a book).

Plaintext

Architecture Component Specifications

1. Presentation Layer (React.js Client)

- **Role:** Handles how the application looks and acts across all client hardware platforms.
- **Target Devices:** Fully responsive optimization ensures seamless performance whether accessed via desktop computers, mid-size tablets, or smartphones.
- **Key Operations:** Renders interactive user interfaces, coordinates book viewing categories, manages client-side form states (login, review inputs), and dispatches network actions via standard REST protocols.

2. Application Layer (Node.js & Express.js)

- **Role:** Serves as the middle logic layer processing client requests and applying backend business logic.
- **Authentication Middleware:** Validates JSON Web Tokens (JWT) to secure checkout, reservations, and review postings.
- **Router Infrastructure:** Segmented API endpoints (/api/books, /api/reviews, /api/orders) parse incoming JSON bodies and direct workflow actions.
- **Recommendation Logic Engine:** Evaluates active user profiles and preferences to map specific database attributes (genre, author, language) into personalized feed suggestions.

3. Data Layer (MongoDB Atlas)

- **Role:** Manages high-performance storage, query handling, and operational record keeping.
- **Users Collection:** Securely catalogs user profile details, login data, and account privileges.
- **Books Collection:** Organizes global data structures for books, tracking explicit metadata strings like genre classifications, specific authors, and language strings.
- **Orders/Reserves Collection:** Coordinates transaction logs, actively monitoring whether an item has been bought outright or held temporarily under a reservation ticket.
- **Reviews Collection:** Houses direct textual feedback, user opinions, and numeric evaluation metrics connected to individual book records.

4. End-to-End Core Data Flow Scenarios

Scenario A: Browsing & Curation Flow

1. Trigger: A user opens BookStore on their mobile device or desktop.
2. Request: The React UI initiates an asynchronous GET request to `/api/books` along with specific query parameters (e.g., genre, author, or language).
3. Processing: Node.js/Express catches the route, queries the MongoDB cluster via Mongoose models, and aggregates the appropriate results.
4. Response: The backend returns a structured JSON payload of matching books. The React frontend stores this in local state and renders the catalog layout responsively.

Scenario B: Transaction Flow (Buy or Reserve)

1. Trigger: A user hits "Buy" or "Reserve" on a chosen book title.
2. Authentication: The frontend embeds a JWT authorization token into the header of a secure POST request sent to `/api/orders`.
3. Processing: The Express middleware unpacks and verifies the token. Upon validation, the server processes the order details and triggers a database transaction.
4. Database Action: MongoDB updates the inventory quantity in the Books Collection and generates a concurrent entry in the Orders/Reserves Collection.
5. Completion: The server confirms success with a 201 Created status code, updating the user's view instantly with their confirmation details.